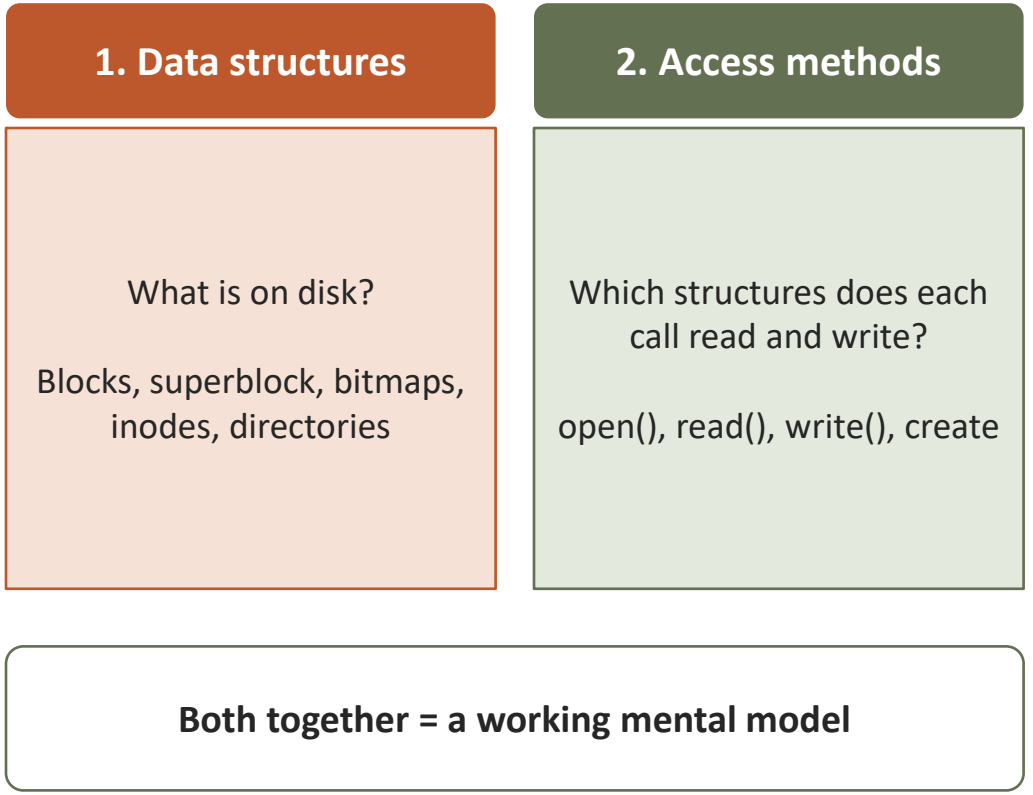
CS3.301: Operating Systems and Networks

File System Implementation

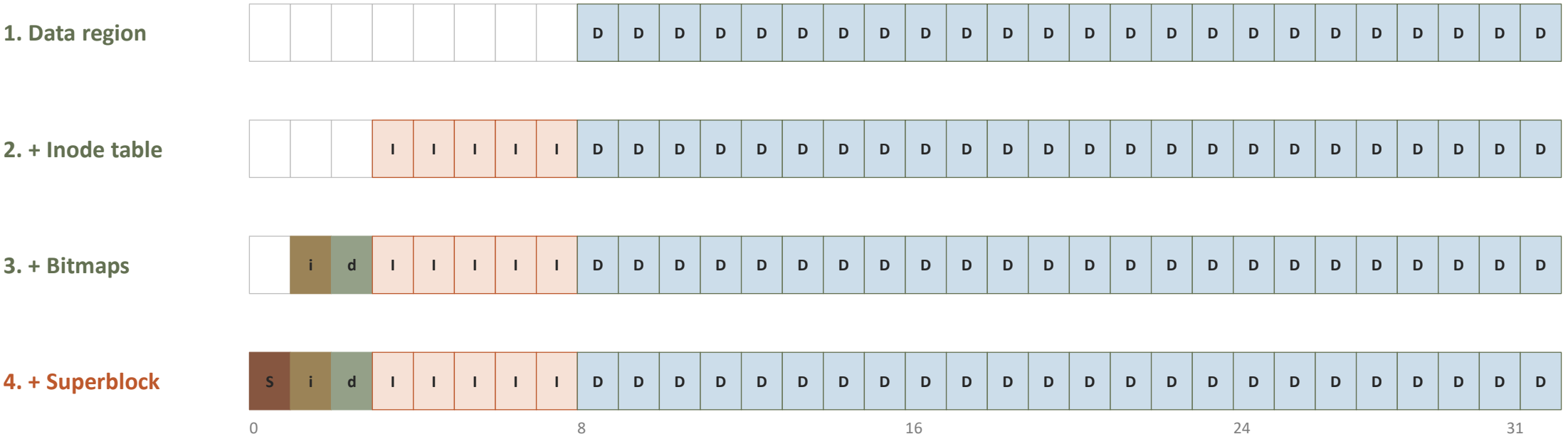
INSTRUCTOR: DR. DEEPAK GANGADHARAN

How to Think About a File System

- Crux: How do we build a simple file system? What on-disk structures are needed, and how are they accessed?
- File systems are **pure software**: no new hardware support
- Many designs exist (AFS to ZFS), so we learn through case studies, starting with **vsfs**
- Goal: a **mental model** of what happens on disk for each call



Simulation: Building the vsfs Layout



64 blocks × 4 KB. Blocks 0–31 shown; 32–63 are data blocks. 256-byte inodes: 16 per 4 KB block × 5 blocks = 80 inodes.

S	superblock
i	inode bitmap
d	data bitmap
I	inode table (80 inodes)
D	data region (56 blocks)

The Superblock and Mounting

- Describes the **whole file system**: counts, locations, type
- Contains a **magic number** identifying the FS type
- On **mount**: read the superblock first, set up parameters, attach the volume to the tree
- Afterwards the OS knows where every other structure lives
- Lose the superblock and nothing else can be found

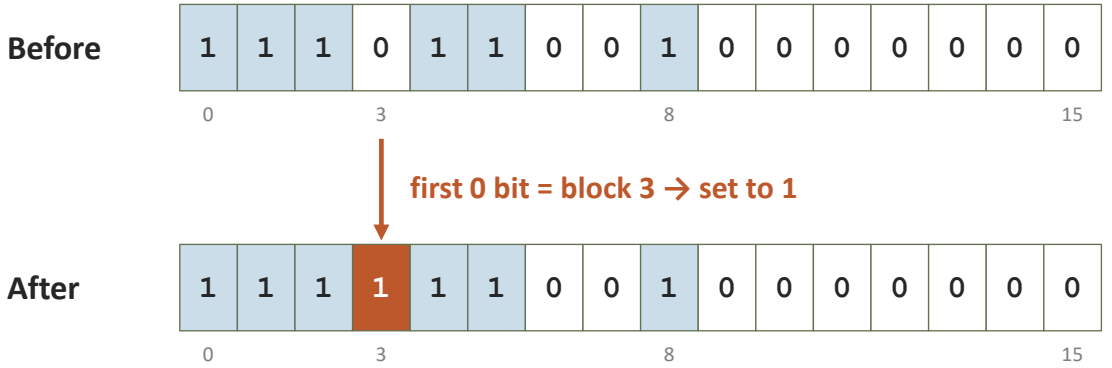
Running example

```
vsfs superblock (block 0)
  magic           = VSFS
  block size      = 4096
  # inodes        = 80
  # data blocks   = 56
  inode table at = block 3
```

Tracking Free Space with Bitmaps

- One bit per object: **0 = free**, **1 = in use**
- Separate **inode bitmap** and **data bitmap**
- Allocate: find a 0, set it to 1, write the bitmap back
- Alternatives: free lists, B-trees (XFS)

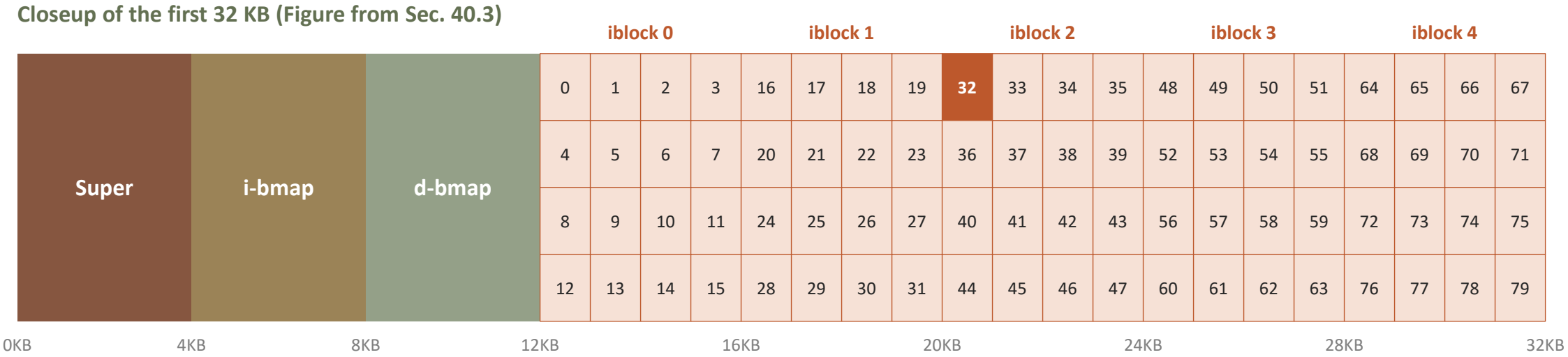
Simulation: allocate one data block



The modified bitmap block must eventually be **written back**: an extra I/O on every allocation

Cost of a bitmap: 1 bit per block.
1 TB disk ÷ 4 KB blocks = 2^28 blocks → 32 MB bitmap

The Inode and the Inode Table



- **Inode** = index node: holds a file's metadata (size, permissions, times, block locations)
- **i-number** = index into the on-disk inode array
- 256-byte inodes → 16 per 4 KB block; 5 blocks → inodes 0–79
- Given an i-number, its disk location is computed, not searched

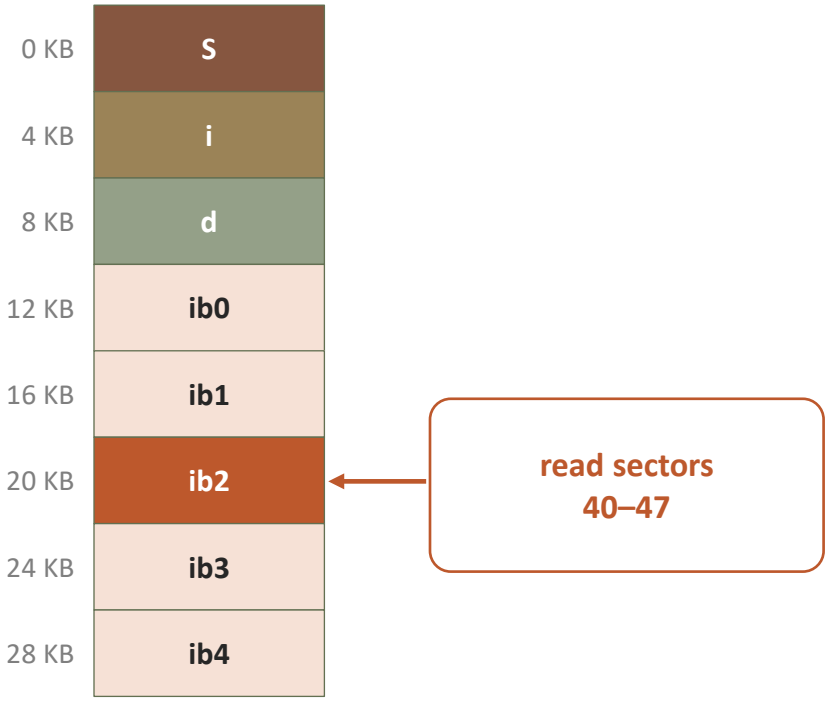
Simulation: Where Is Inode 32 on Disk?

General formula

```
blk    = (inumber * sizeof(inode_t)) / blockSize;
sector = ((blk * blockSize) + inodeStartAddr) / sectorSize;
```

Step	Inode 32	Inode 70 (try it)
1. Offset in table = i × 256	8192 B	17920 B
2. Inode block = offset ÷ 4096	iblock 2	iblock 4
3. Disk address = 12 KB + blk × 4 KB	20 KB	28 KB
4. Sector = address ÷ 512	40	56
5. Offset within block = (i mod 16) × 256	0 B	1536 B

Locating iblock 2 on disk



Disks address 512-byte sectors, not bytes

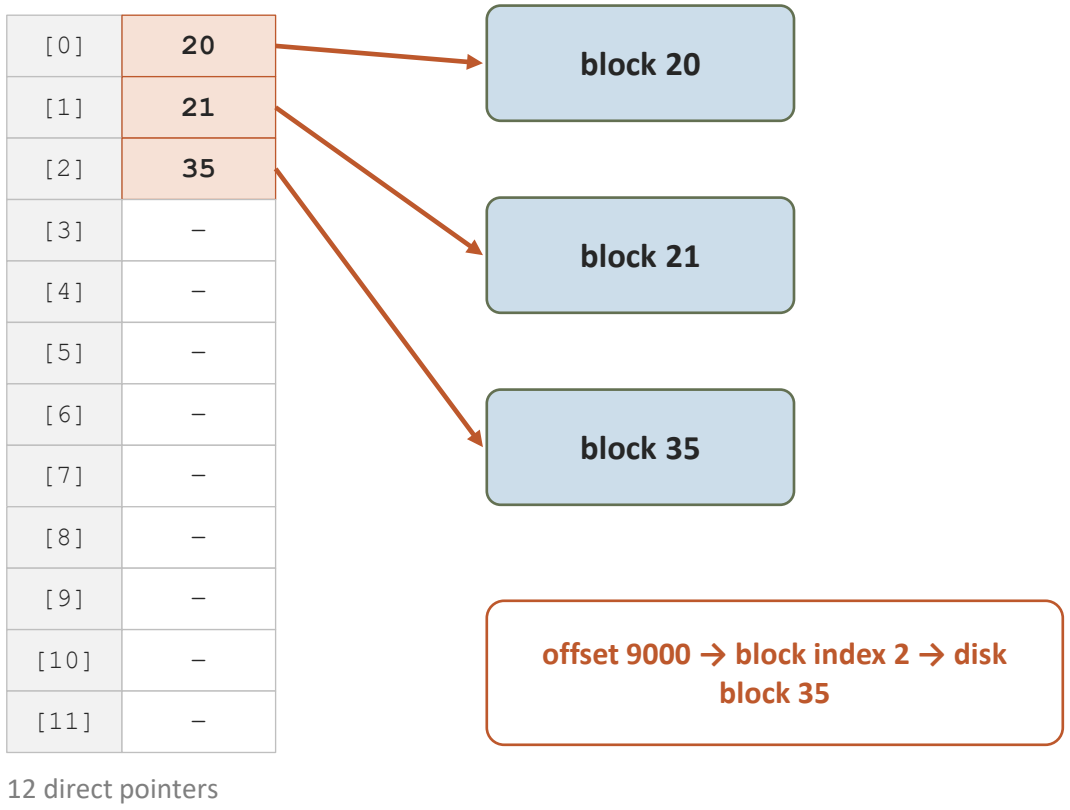
What Is Inside an Inode?

Size (B)	Field	Purpose
2	mode	read/write/execute permissions
2	uid	owner
4	size	bytes in the file
4 × 4	time, ctime, mtime, dtime	access, inode change, modify, delete times
2	gid	group
2	links count	number of hard links
4	blocks	blocks allocated
4	flags / osd1	how ext2 uses the inode; OS-specific
60	block	15 disk pointers to the data
4 × 3	generation, file acl, dir acl	NFS version; ACLs

- Metadata = everything that is **not** user data
- `stat()` reads these fields
- The key design choice: how the **block** pointers find the data

Direct Pointers and Their Limit

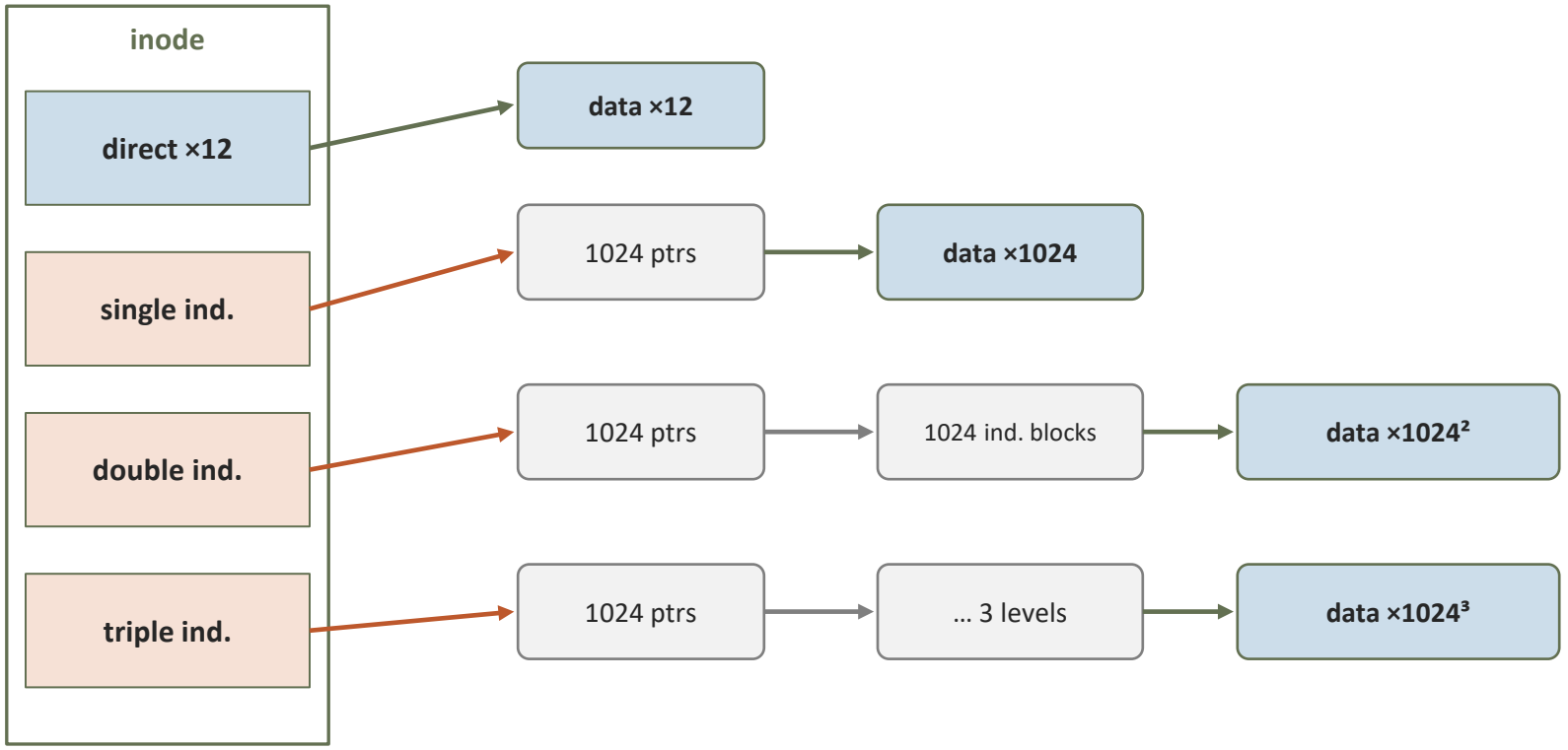
Inode of a 10 KB file (3 blocks)



12 direct pointers

- **Direct pointer** = disk address of one data block
- ext2: 12 direct pointers
- Max size = 12 × 4 KB = **48 KB**
- Blocks need not be contiguous
- Bigger file? Out of luck, unless we add indirection

The Multi-Level Index



- **Indirect** pointer → a block of 1024 pointers (4 KB ÷ 4 B)
- **Double / triple** indirect add 1024^2 and 1024^3 blocks
- ext2: 12 + 1 + 1 + 1 = **15** pointers
- An imbalanced tree: small files stay shallow

Simulation: File Size Limits and a Block Lookup

Max file size (4 KB blocks, 4 B pointers)

Pointers used	Blocks	Max size
12 direct	12	48 KB
+ single indirect	12 + 1024	≈ 4 MB
+ double indirect	+ 1024^2	≈ 4 GB
+ triple indirect	+ 1024^3	≈ 4 TB

Each extra level multiplies capacity by **1024**

Which pointer serves byte offset 5,000,000?

1. Block index = 5,000,000 ÷ 4096 = **1220**
2. 1220 ≥ 12 → not direct; 1220 − 12 = 1208
3. 1208 ≥ 1024 → not single; 1208 − 1024 = 184
4. Double indirect: outer slot 0, inner slot **184**
5. Disk reads: inode → dbl block → ind. block → data

Why an Imbalanced Tree? Most Files Are Small

Finding	Typical value
Most files are small	≈ 2 KB is the most common size
Average file size is growing	Almost 200 KB average
Most bytes are in large files	A few big files use most of the space
File systems contain lots of files	Almost 100K on average
File systems are roughly half full	≈ 50% full even as disks grow
Directories are typically small	Most have 20 or fewer entries

Source: Agrawal et al., FAST '07 (Figure 40.2)

12 direct pointers
= 48 KB

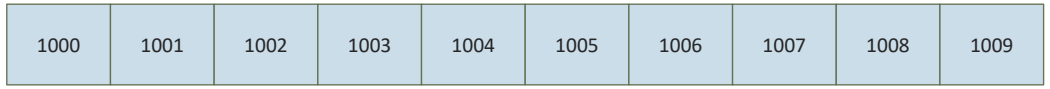
covers the common small file with **no** extra reads

Optimize the common case;
keep the rare case possible

An Alternative: Extents

Running example: a 1.2 MB file stored in blocks 1000–1299 (300 blocks)

Pointer-based



… 300 pointers (12 direct + indirect block)

Extent-based

```
start = 1000
length = 300
```

1 entry describes the whole file

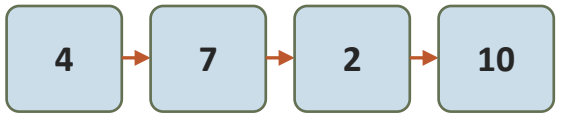
	Pointers (ext2, ext3)	Extents (ext4, XFS)
Metadata per file	One pointer per block: large for big files	One (start, length) per contiguous run
Flexibility	Any block anywhere	Needs contiguous free space
VM analogy	Paging	Segmentation

Another Alternative: Linked Allocation and FAT

File Allocation Table (kept in memory)

FAT[0]	–
FAT[1]	–
FAT[2]	**10**
FAT[3]	free
FAT[4]	**7**
FAT[5]	free
FAT[6]	–
FAT[7]	**2**
FAT[8]	free
FAT[9]	–
FAT[10]	**EOF**
FAT[11]	free

```
dir entry: "a.txt"
first block = 4
```



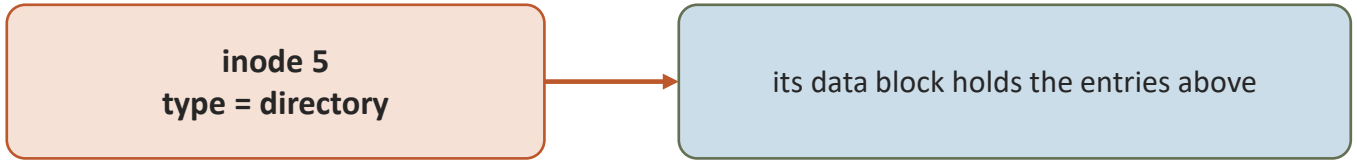
file = blocks 4 → 7 → 2 → 10

- **Linked list**: inode points to first block; each block points to the next
- Random access / last block = walk the whole chain
- **FAT** moves next-pointers into an in-memory table
- No inodes: metadata lives in directory entries
- Consequence: hard links are impossible in FAT

Directory Organization

Data block of directory dir (inode 5)

inum	reclen	strlen	name
5	12	2	.
2	12	3	..
12	12	4	foo
13	12	4	bar
24	36	28	foobar_is_a_pretty_longname

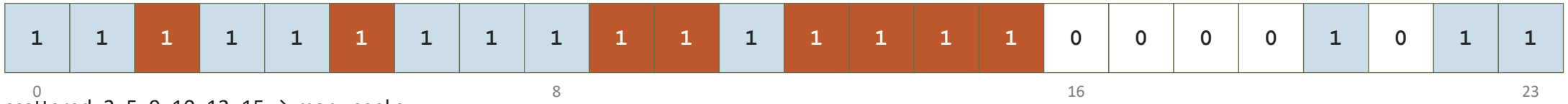


- Entry = (inode number, record length, name length, name)
- `.` = this directory, `..` = parent
- unlink() leaves a hole (inum 0); **reclen** lets a new entry reuse it
- A directory is a **file** of type "directory": inode + data blocks
- Linear list is simple; XFS uses **B-trees** for fast lookups

Free-Space Policy: Pre-allocation

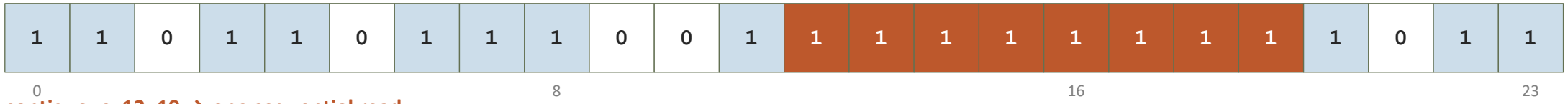
Data bitmap (blocks 0–23); new file needs 8 blocks

First-fit



scattered: 2, 5, 9, 10, 12–15 → many seeks

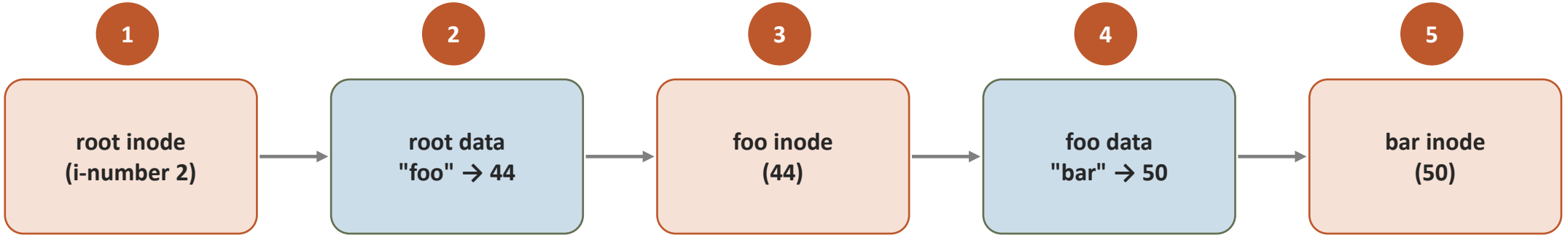
Pre-allocate 8 contiguous



contiguous: 12–19 → one sequential read

- Create: find a free inode in the inode bitmap; allocate data blocks from the data bitmap
- ext2/ext3 heuristic: look for a run of ~8 free blocks so part of the file is **contiguous**

Simulation: open("/foo/bar") Walks the Path



inode → data → inode → data → inode: each level costs ≥ 2 reads

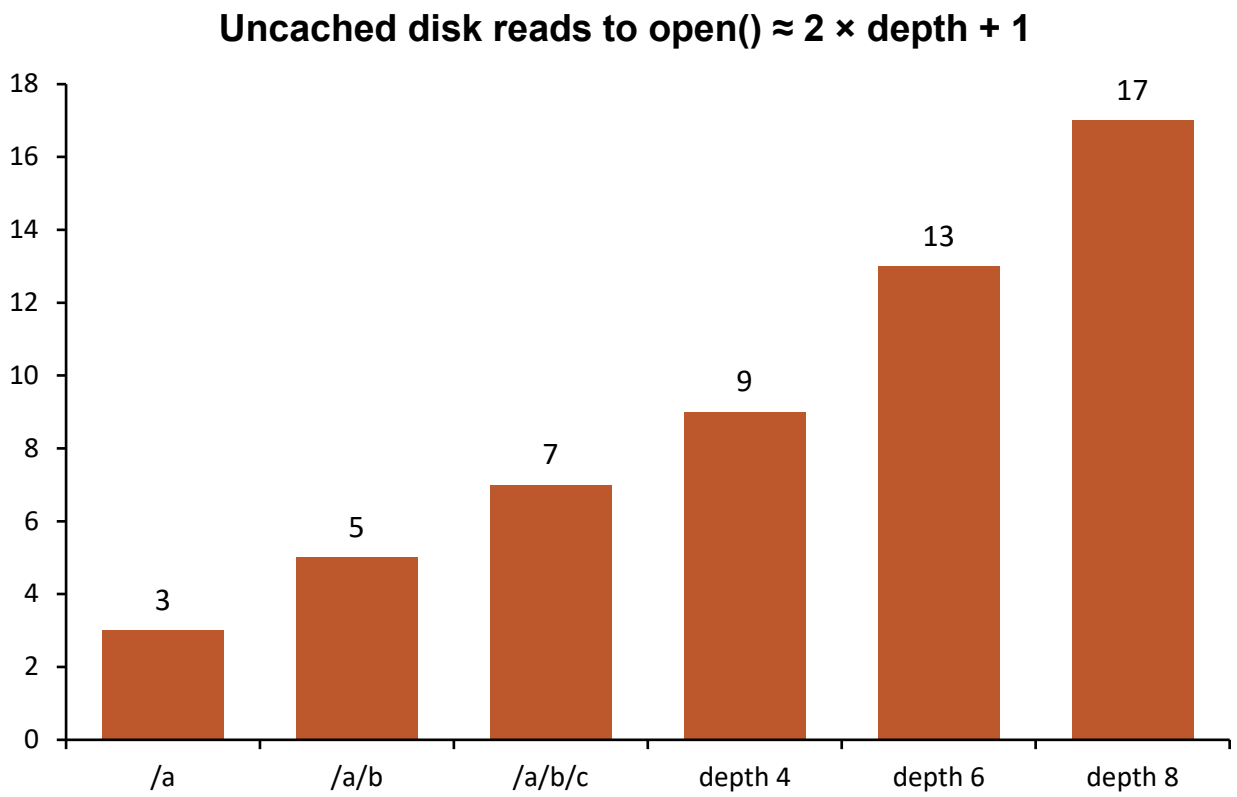
- Assume mounted: **superblock in memory**, everything else on disk
- Root's i-number is **well known** (2 in most UNIX FSes): it has no parent to look it up in
- Directory data maps each name to the next i-number
- Finally: permission check, allocate an fd and an open-file-table entry, return
- Cost of open() grows with path length (and directory size)

File Read Timeline (Figure 40.3)

	data bmap	inode bmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
open(bar)			R1	R3	R5	R2	R4			
read()					R1 W3			R2		
read()					R1 W3				R2	
read()					R1 W3					R2
close()										

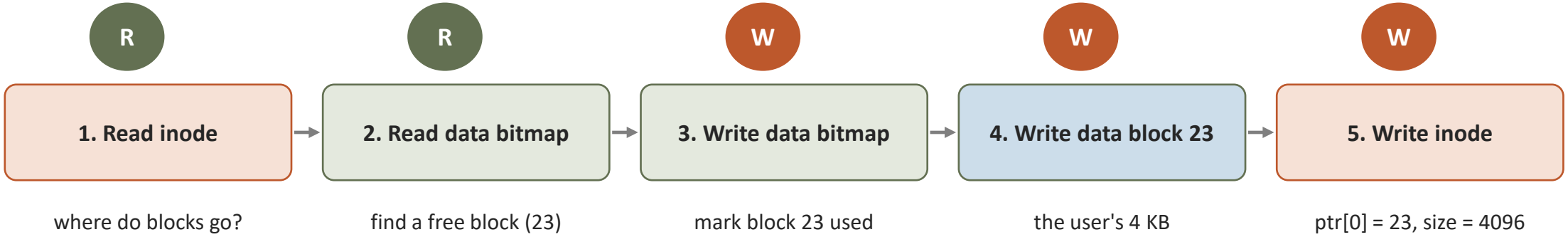
- open: 5 reads; each read(): read inode, read data, **write inode** (access time)
- close(): no I/O. Total: 5 + 3 × 3 = **14 I/Os**
- Reads never touch the bitmaps

The Cost of Long Pathnames



- Each directory level: read its **inode** + its **data**
- Large directories: even more data blocks to scan
- `/1/2/…/100/file.txt` → hundreds of reads
- Motivates caching directory and inode blocks

Writing a File: 5 I/Os per Allocating Write



Only 1 of the 5 I/Os is user data; the other 4 are metadata

- Overwriting an existing block needs no allocation (no bitmap I/O)
- A new file of N blocks: ≈ 5N I/Os for the writes alone, plus the cost of creating it

File Creation Timeline (Figure 40.4)

	data bmap	inode bmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
create (/foo/bar)		R5 W6	R1	R3 W10	R8 W9	R2	R4 W7			
write()	R2 W3				R1 W5			W4		
write()	R2 W3				R1 W5				W4	
write()	R2 W3				R1 W5					W4

- create = **10 I/Os**: walk path, allocate inode, add dir entry, init inode, update dir inode
- Why read the brand-new inode? It shares a 4 KB block: **read-modify-write**
- Total: 10 + 3 × 5 = 25 I/Os to create a 12 KB file

Simulation (vsfs.py): mkdir and creat

State	Inode bmap	Inodes	Data bmap	Data blocks
`initial`	1000	[d a:0 r:2] [] [] []	1000	[(.,0) (..,0)] [] [] []
`mkdir("/g")`	1100	[d a:0 r:3] [d a:1 r:2] [] []	1100	[(.,0) (..,0) (g,1)] [(.,1) (..,0)] [] []
`creat("/g/q")`	1110	[d a:0 r:3] [d a:1 r:2] [f a:-1 r:1] []	1100	[(.,0) (..,0) (g,1)] [(.,1) (..,0) (q,2)] [] []

- Notation: [type a:data-block r:link-count]; d = directory, f = file
- mkdir allocates an **inode and a data block**; parent's link count +1 (child's `..`)
- creat allocates **only an inode** (a:-1): no data until the first write
- Predict each change before revealing it (homework Q2)

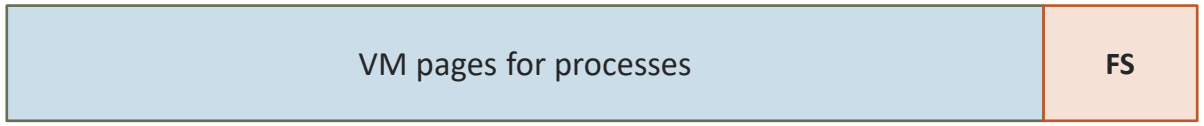
Simulation (vsfs.py): write, link and unlink

State	Inode bmap	Inodes	Data bmap	Data blocks
`write /g/q`	1110	[d a:0 r:3] [d a:1 r:2] [f a:2 r:1] []	1110	[(.,0) (..,0) (g,1)] [(.,1) (..,0) (q,2)] [m] []
`link → /w`	1110	[d a:0 r:3] [d a:1 r:2] [f a:2 r:2] []	1110	[(.,0) (..,0) (g,1) (w,2)] [(.,1) (..,0) (q,2)] [m] []
`unlink /g/q`	1110	[d a:0 r:3] [d a:1 r:2] [f a:2 r:1] []	1110	[(.,0) (..,0) (g,1) (w,2)] [(.,1) (..,0)] [m] []

- First write allocates a **data block**: data bitmap + inode + block change
- link adds a **name** only: directory entry + link count
- unlink removes a name; inode and block are freed only when the count hits **0**
- Allocator prefers the lowest free numbers

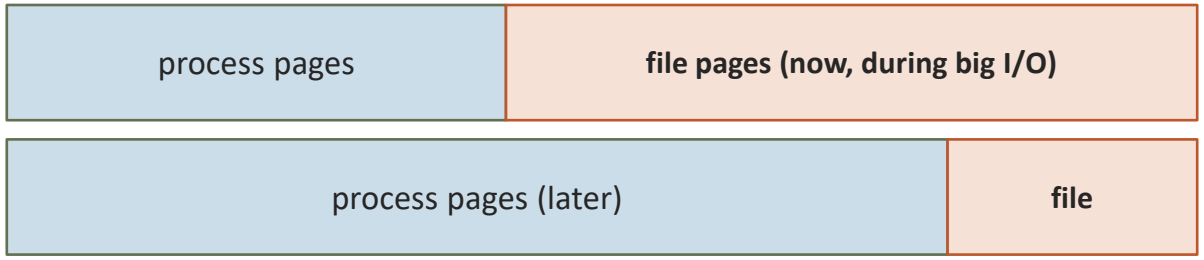
Reducing I/O: Caching and the Page Cache

Static: fixed file cache (≈10% of RAM)



idle cache pages are wasted; can't grow when needed

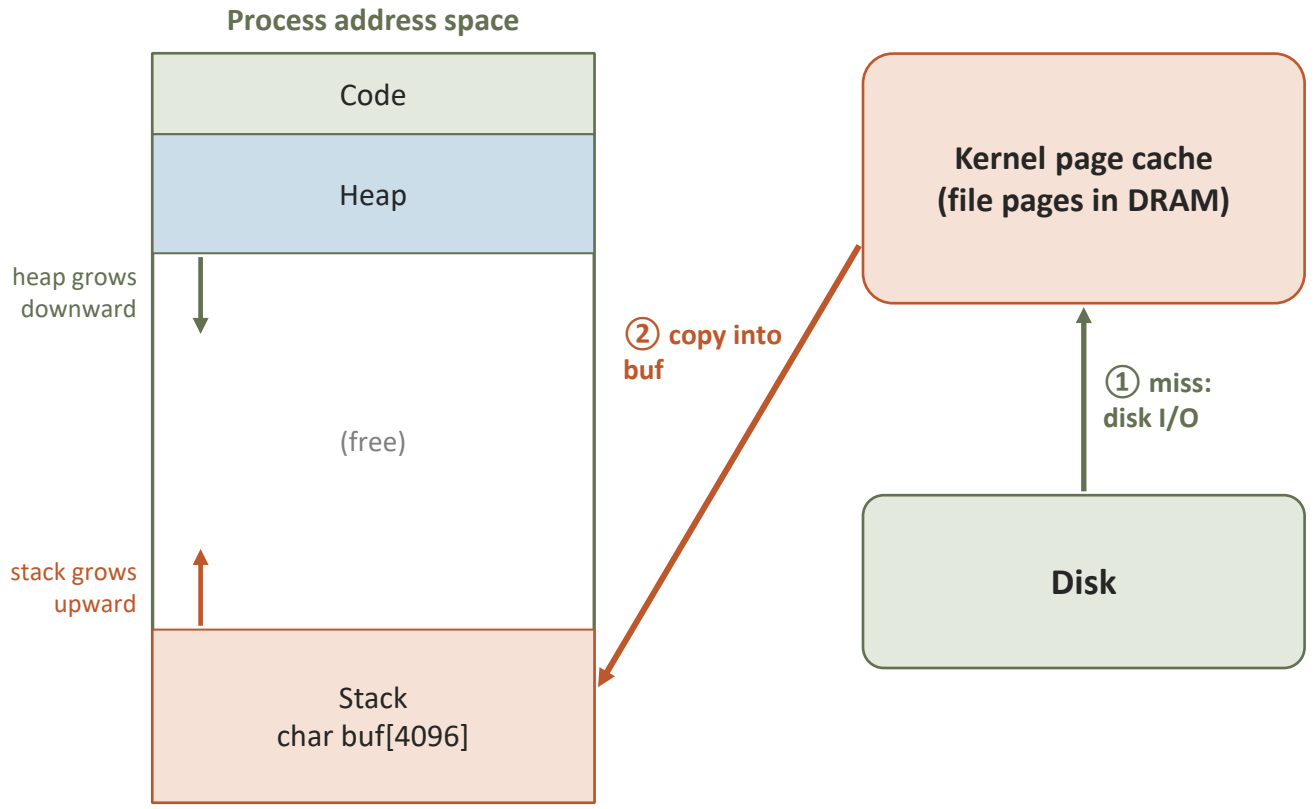
Dynamic: unified page cache



the boundary moves with demand

- Crux: how to reduce file-system I/O costs?
- Cache popular blocks in **DRAM** (LRU-like replacement)
- First open: many I/Os; later opens: **cache hits**
- Static partitioning: predictable, but wastes idle memory
- Dynamic (unified page cache): better utilization

Inside read(): The Page Cache and Your Buffer



- **Miss**: disk → page cache → copy into buf
- **Hit**: page cache → copy into buf, no disk I/O
- Cached pages are shared by all processes

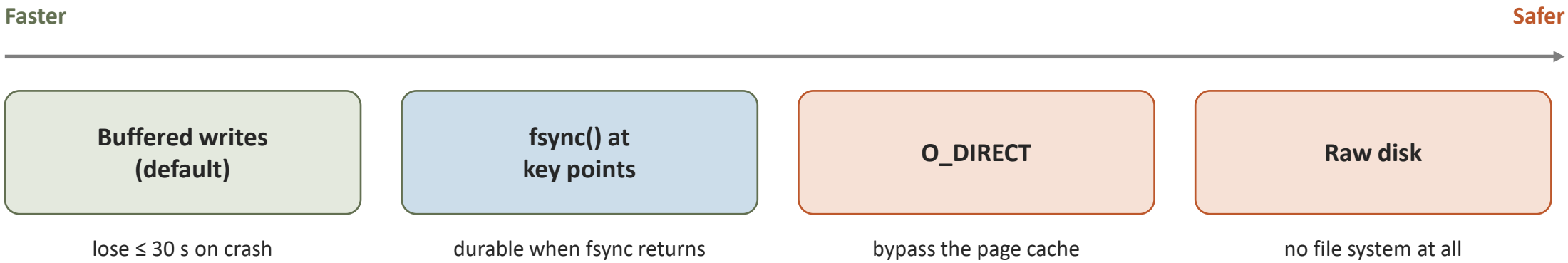
Write Buffering: Batch, Schedule, Avoid

Simulation: three scenarios inside one 30-second buffering window

Scenario	Write immediately	Buffer and write later	Benefit
create a.txt, then b.txt (same inode bitmap block)	2 bitmap writes	1 bitmap write	Batching
100 dirty blocks scattered across the disk	100 writes in arrival order	100 writes, sorted by location	Scheduling
create tmp file, write it, delete it	≈ 10+ I/Os	0 I/Os	Avoidance

- Caching absorbs reads; writes must **eventually** reach disk
- File systems typically buffer writes **5–30 s**
- Risk: a crash before writeback loses those updates

The Durability / Performance Trade-off



Application	Acceptable loss?	Sensible choice
Web browser cache	Last few images: fine	Buffered writes
Bank database transaction	None	fsync() / O_DIRECT

Try It: vsfs Ideas on a Real Linux System

Beyond the textbook

Command	Shows	vsfs concept
`ls -i file ; stat file`	inode number, links, size, blocks	inode, i-number
`ls -id /`	root inode = 2	well-known root i-number
`df -i`	inodes used / free	fixed-size inode table
`sudo dumpe2fs -h /dev/sda1`	superblock fields, magic 0xEF53	superblock
`filefrag -v file`	extents (start, length)	extents vs pointers
`sudo debugfs -R 'stat <12>' /dev/sda1`	raw inode fields + extent list	Figure 40.1

Summary and Homework

Superblock	Bitmaps	Inode	Multi-level index
FS-wide parameters; read first at mount	Free/used bit per inode and per data block	Per-file metadata; location computed from i-number	12 direct + single/double/triple indirect

Directory	Access paths	Page cache	Write buffering
A file of (name, i-number) entries	open ≈ 2 reads/level; allocating write = 5 I/Os	Absorbs reads; unified with VM	Batch, schedule, avoid; fsync for durability

Thank You!!